

GREAT LEARNING

CAPSTONE PROJECT REPORT

ON

DOCONNECT AI

An Intelligent Community Platform for Knowledge Sharing and Collaboration

**Submitted in partial fulfillment of the requirements for the
Capstone Project Program**

Submitted By

Panuganti Madhusudan

Project Engineer

Wipro Limited

Submitted To

Great Learning

Month Year

(June 2026)

Declaration

I hereby declare that the project titled "DoConnect AI – An Intelligent Community Platform for Knowledge Sharing and Collaboration" submitted to Great Learning is my original work and has not been submitted elsewhere for any degree, diploma, or certification. All sources of information used in this project have been duly acknowledged.

Signature:

Panuganti Madhusudan

Date: 18-06-2026

Place: Hyderabad

TABLE OF CONTENTS

CHAPTER 1: BUSINESS DOCUMENTATION

- 1.1 Problem Statement
- 1.2 Objectives
- 1.3 User Stories
- 1.4 Use Cases
- 1.5 Scope of the Project

CHAPTER 2: TECHNICAL DOCUMENTATION

- 2.1 System Architecture Diagram
- 2.2 Use Case Diagram
- 2.3 Class Diagram
- 2.4 Sequence Diagram
- 2.5 Data Flow Diagram (DFD)
- 2.6 Layered Architecture Diagram
- 2.7 Entity Relationship (ER) Diagram
- 2.8 Database Design
 - 2.8.1 Users Table
 - 2.8.2 Questions Table
 - 2.8.3 Answers Table
 - 2.8.4 Comments Table
 - 2.8.5 Votes Table
 - 2.8.6 Chat Rooms Table
 - 2.8.7 Chat Messages Table
 - 2.8.8 Database Relationships
 - 2.8.9 Database Normalization

2.9 API Documentation

2.9.1 Authentication APIs

2.9.2 Question APIs

2.9.3 Answer APIs

2.9.4 Comment APIs

2.9.5 Vote APIs

2.9.6 Chat Room APIs

2.9.7 Chat Message APIs

2.10 Deployment Guide

2.10.1 Software Requirements

2.10.2 Backend Setup

2.10.3 Frontend Setup

2.10.4 Database Setup

2.11 Folder Structure

CHAPTER 3: TESTING DOCUMENTATION

3.1 Test Cases

3.2 Execution Results

3.3 Defect Summary

References

CHAPTER 1: BUSINESS DOCUMENTATION

1.1 Problem Statement

In today's learning environment, students and developers frequently encounter technical challenges while working on projects, assignments, and new technologies. Existing discussion platforms often lack real-time collaboration, organized knowledge sharing, integrated communication features, and personalized assistance. Users typically rely on multiple platforms for asking questions, participating in discussions, chatting with peers, and seeking technical guidance, which leads to fragmented learning experiences.

There is a need for a unified platform that enables users to ask questions, share knowledge, collaborate through chat rooms, receive notifications, and interact with intelligent assistance features in a secure and user-friendly environment.

The proposed solution, DoConnect AI, aims to address these challenges by providing an intelligent community platform that integrates question-answer discussions, chat functionality, notifications, role-based access control, and AI-powered assistance into a single web application.

1.2 Objectives

The primary objective of the DoConnect AI project is to develop an intelligent community platform that enables students and developers to collaborate, share knowledge, and seek technical assistance through an integrated web-based application.

The specific objectives of the project are:

- To provide a secure user authentication system using JWT-based authentication and role-based authorization.
- To enable users to post technical questions and share answers to facilitate collaborative learning and knowledge sharing.
- To provide interactive chat rooms where users can communicate and discuss technology-related topics in real time.
- To implement a notification system that informs users about important activities and updates within the platform.
- To provide an intuitive and user-friendly interface using React for improved user experience and easy navigation.
- To design a normalized and efficient database structure using MySQL and JPA relationships for maintaining data consistency and integrity.

- To implement robust validation, exception handling, and secure coding practices following industry standards.
- To create a scalable and maintainable full-stack application by following layered architecture and REST API design principles.
- To integrate intelligent assistance features that help users stay informed and improve their learning experience.
- To provide a centralized platform that combines question-answer discussions, communication, and collaboration within a single application.

1.3 User Stories

User Story ID	Actor	User Story
US-01	User	As a user, I want to register an account so that I can access the platform features.
US-02	User	As a user, I want to log in securely so that I can use my account safely.
US-03	User	As a user, I want to post technical questions so that I can seek solutions from the community.
US-04	User	As a user, I want to answer questions so that I can share my knowledge with others.
US-05	User	As a user, I want to comment on answers so that I can provide additional information and feedback.
US-06	User	As a user, I want to vote on answers so that useful answers can be identified easily.
US-07	User	As a user, I want to join chat rooms so that I can discuss technology-related topics with others.
US-08	User	As a user, I want to receive notifications so that I can stay updated with platform activities.
US-09	User	As a user, I want to use AI assistance so that I can obtain quick guidance and suggestions.
US-10	Admin	As an admin, I want to manage users so that I can maintain platform security and integrity.
US-11	Admin	As an admin, I want to manage questions and answers so that inappropriate content can be controlled.

User Story ID	Actor	User Story
US-12	Admin	As an admin, I want to manage chat rooms so that discussions remain organized and relevant.

1.4 Use Cases

User Use Cases

Use Case ID	Use Case Name	Actor	Description
UC-01	Register	User	Creates a new account.
UC-02	Login	User	Logs into the application.
UC-03	Ask Question	User	Posts a technical question.
UC-04	View Questions	User	Views all questions.
UC-05	Post Answer	User	Adds an answer to a question.
UC-06	Comment	User	Adds comments on answers.
UC-07	Vote	User	Upvotes or downvotes answers.
UC-08	Join Chat Room	User	Participates in chat discussions.
UC-09	View Notifications	User	Receives notifications.
UC-10	Use AI Assistant	User	Uses AI features for assistance.

Admin Use Cases

Use Case ID	Use Case Name	Actor	Description
UC-11	Manage Users	Admin	Views and manages users.
UC-12	Manage Questions	Admin	Deletes or manages questions.
UC-13	Manage Answers	Admin	Deletes or manages answers.
UC-14	Manage Chat Rooms	Admin	Creates and deletes chat rooms.

1.5 Scope of Project

The scope of the DoConnect AI project is to develop a web-based community platform that facilitates knowledge sharing, collaboration, and communication among students and developers. The application provides a centralized environment where users can ask questions, post answers, participate in discussions, and receive updates regarding platform activities.

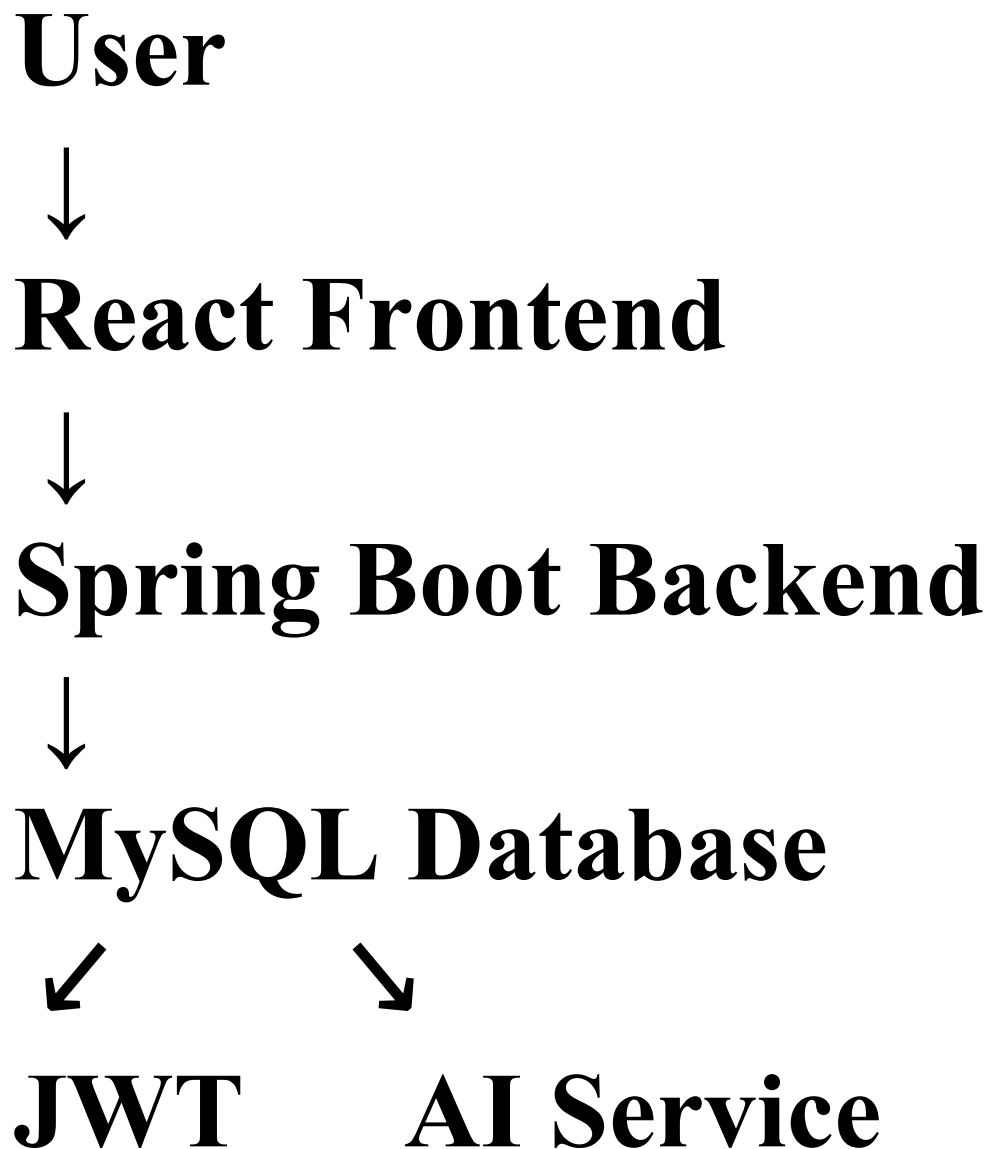
The project includes the following functionalities:

- User registration and secure login using JWT authentication.
- Role-based access control for Users and Administrators.
- Posting and managing technical questions and answers.
- Commenting and voting on answers to encourage community participation.
- Creation and management of chat rooms for topic-based discussions.
- Sending and receiving notifications related to user activities.
- Administrative features for managing users, questions, answers, and chat rooms.
- Integration of intelligent assistance features to enhance the learning experience.
- Responsive and user-friendly interfaces developed using React.
- Backend services developed using Spring Boot REST APIs with MySQL database integration.

The project is intended for educational and collaborative purposes and can be further extended by incorporating advanced features such as real-time WebSocket communication, recommendation systems, mobile application support, and enhanced AI-driven functionalities.

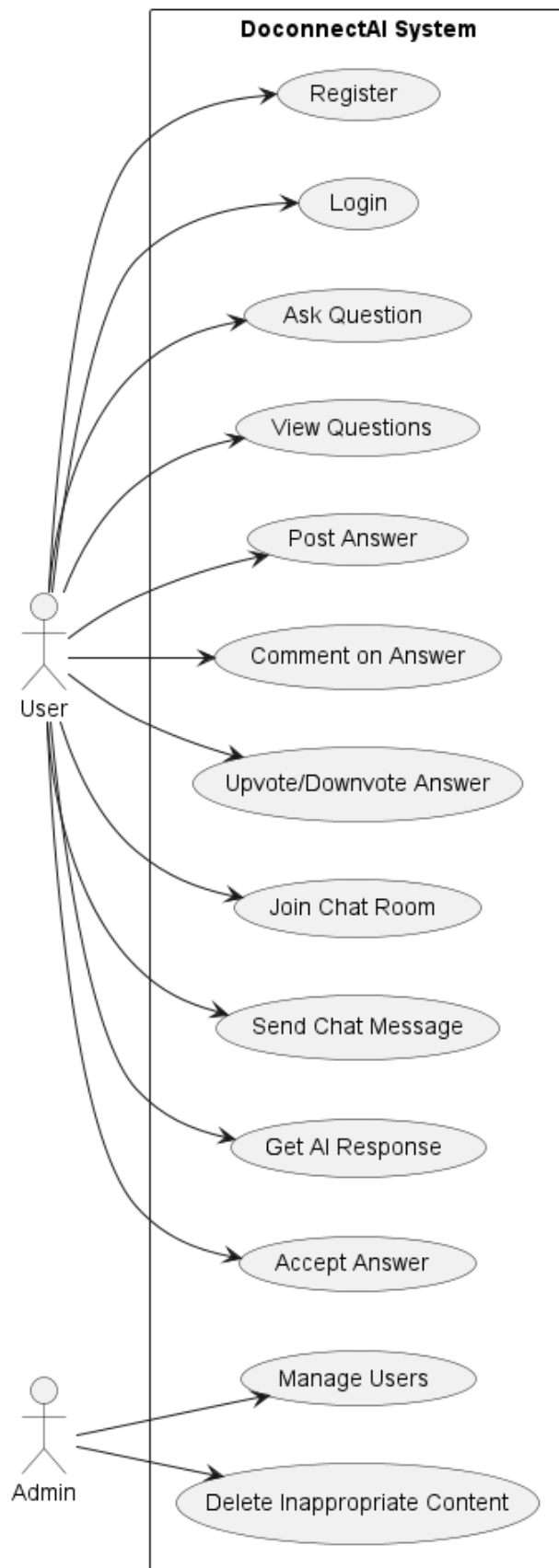
CHAPTER 2: TECHNICAL DOCUMENTATION

2.1 System Architecture Diagram

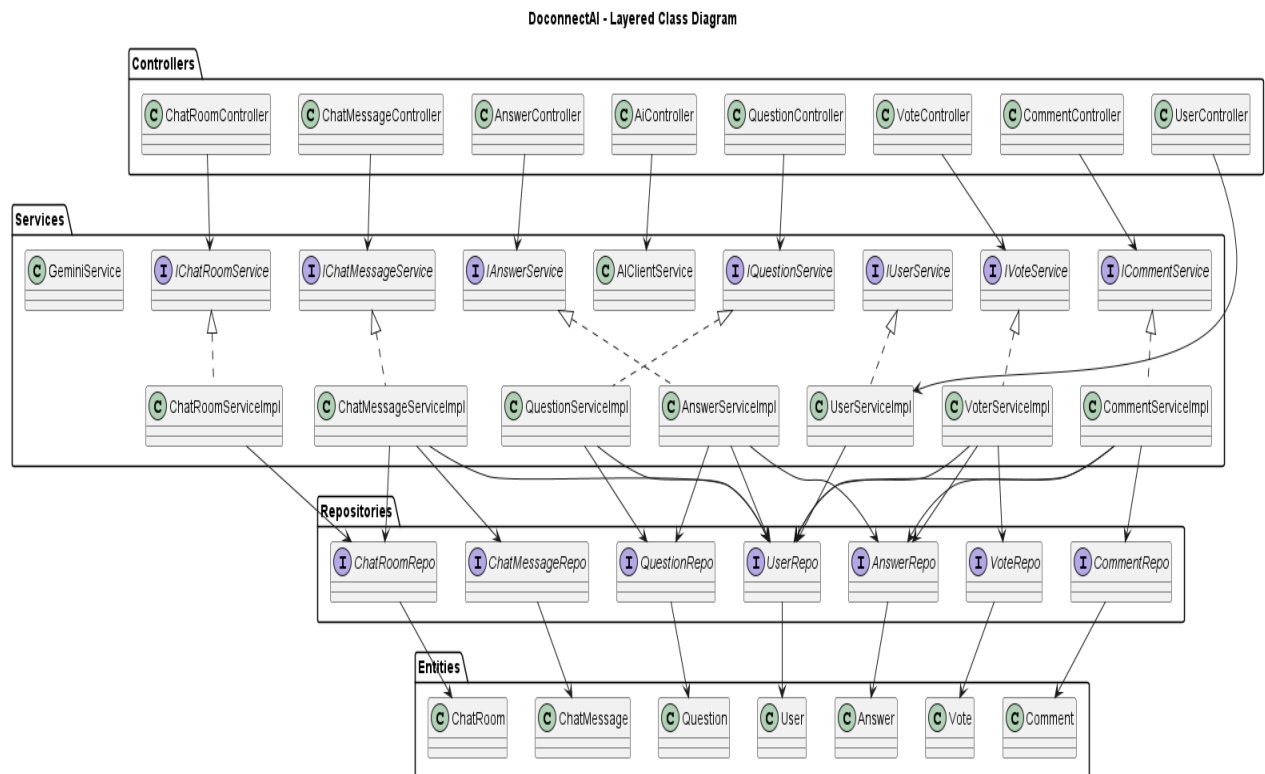


The DoConnect AI application follows a three-tier architecture consisting of a React-based frontend, Spring Boot REST APIs as the backend, and MySQL as the database layer. Users interact with the system through the React user interface, which communicates with the backend services using REST APIs. The backend handles business logic, authentication, validation, and database operations. JWT-based security mechanisms are used for authentication and authorization, while the AI service integrates intelligent assistance features. The MySQL database stores all application data, including users, questions, answers, comments, votes, and chat information.

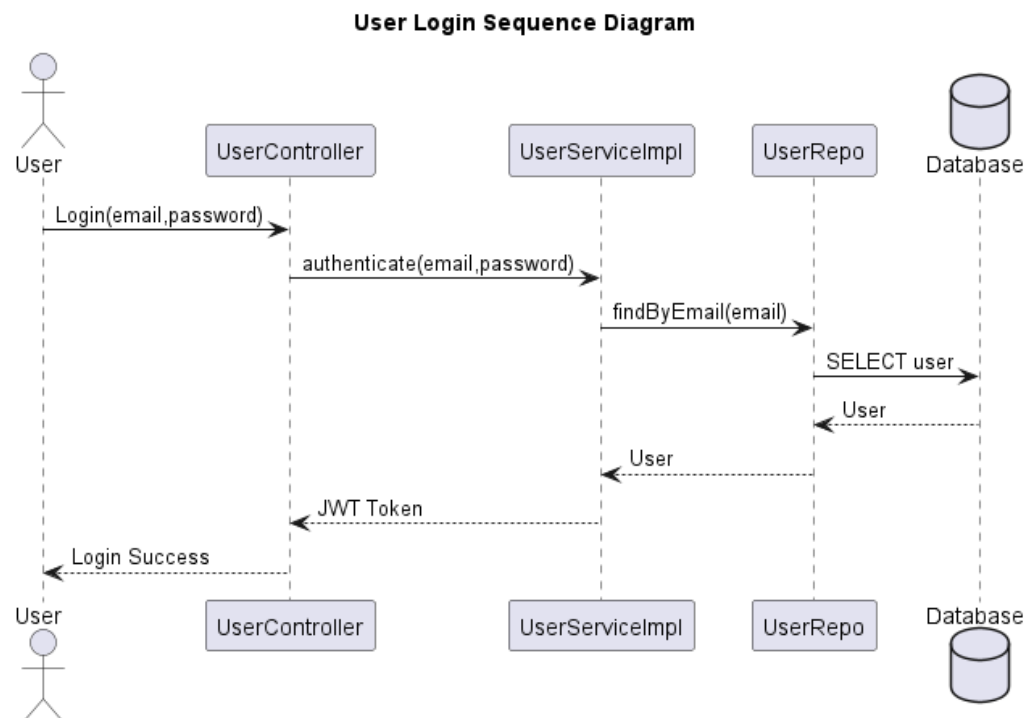
2.2 Use Case Diagram



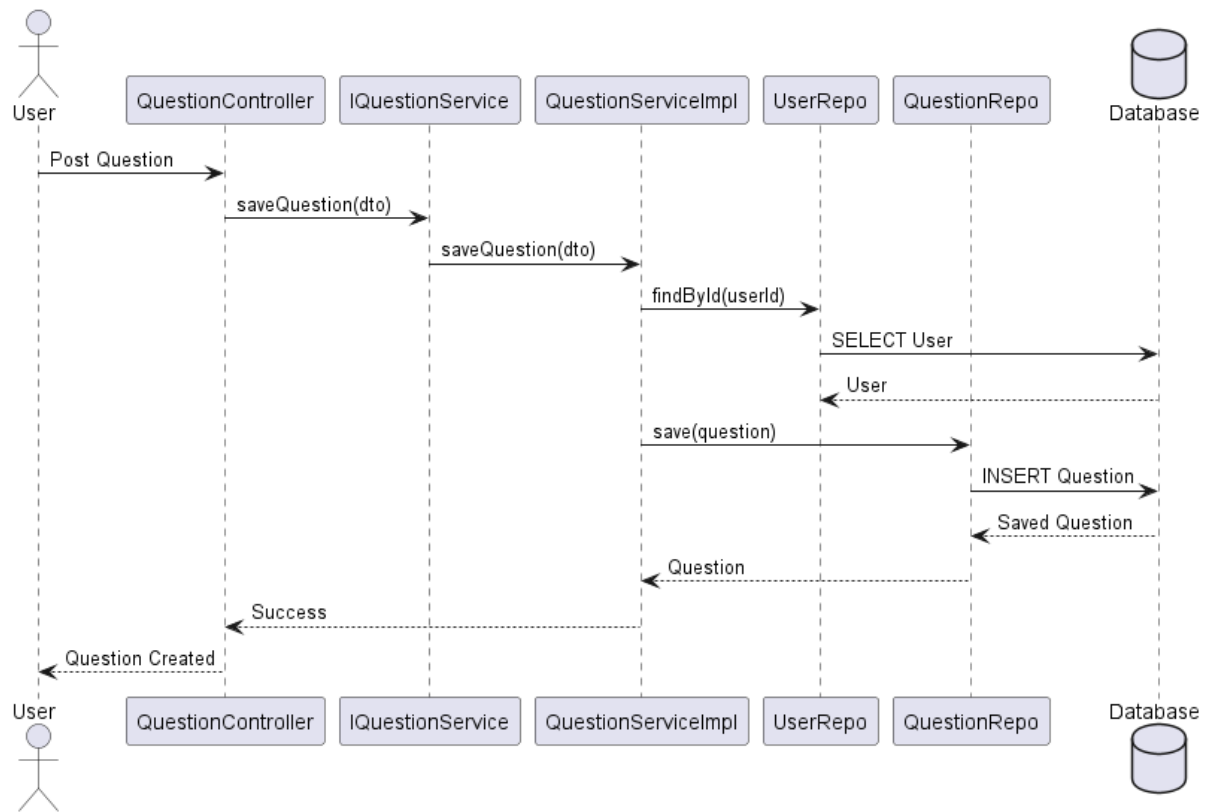
2.3 Class Diagram



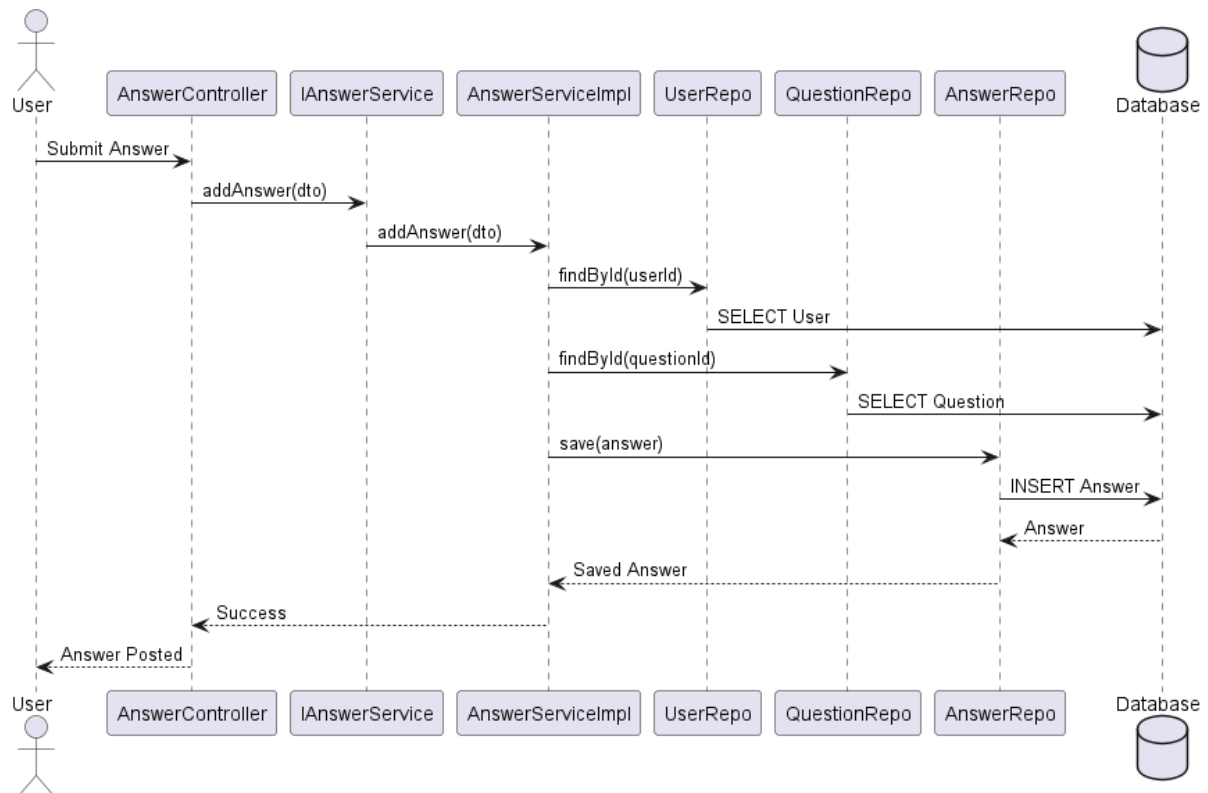
2.4 Sequence Diagram



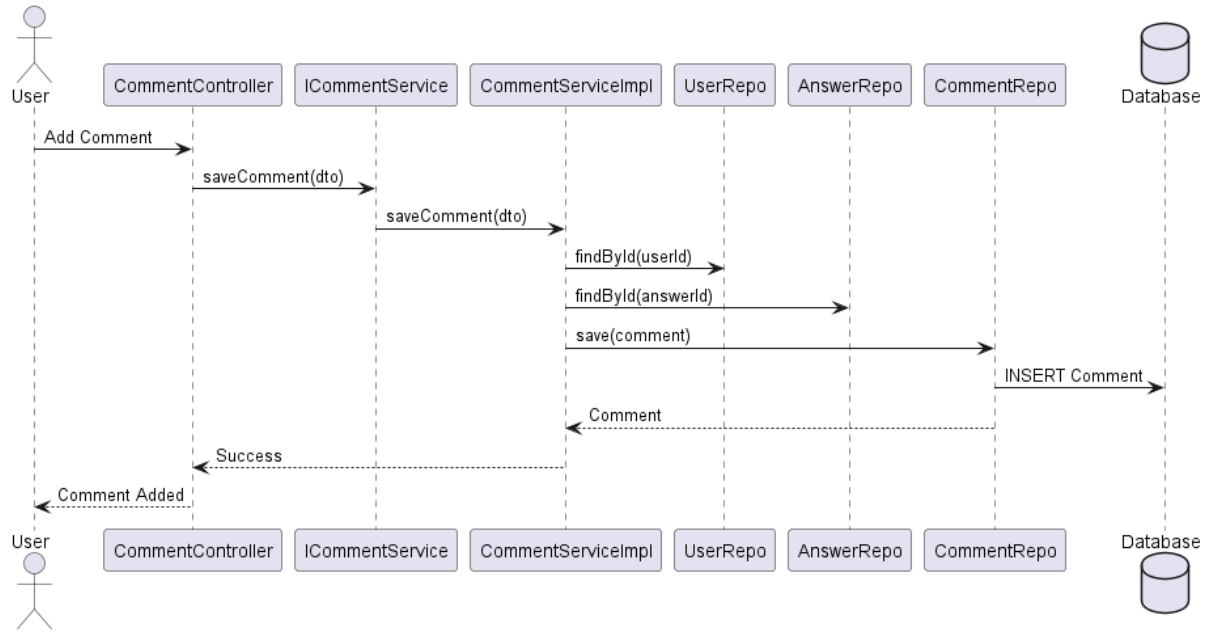
Ask Question Sequence Diagram



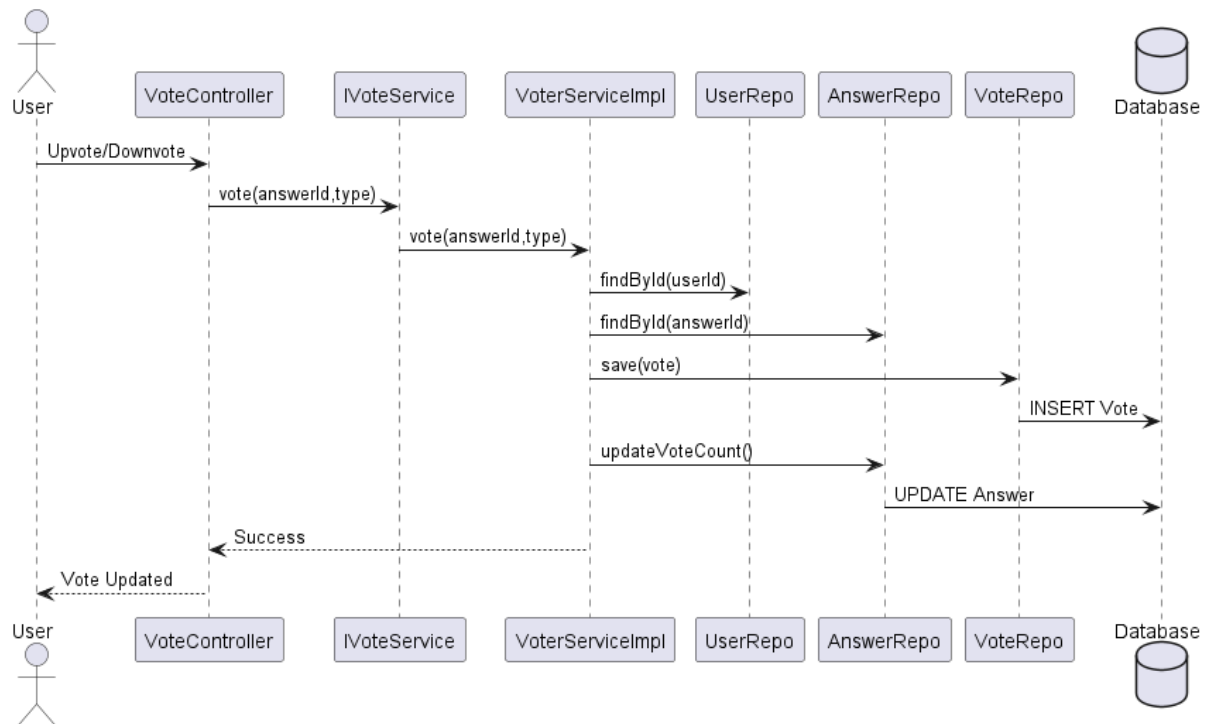
Post Answer Sequence Diagram

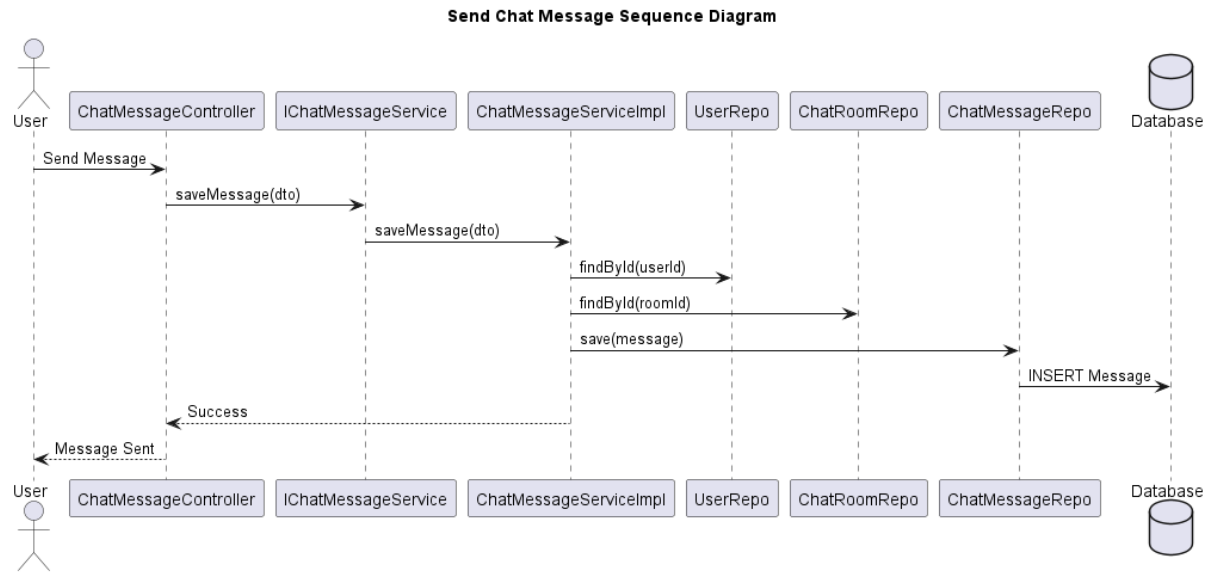


Add Comment Sequence Diagram

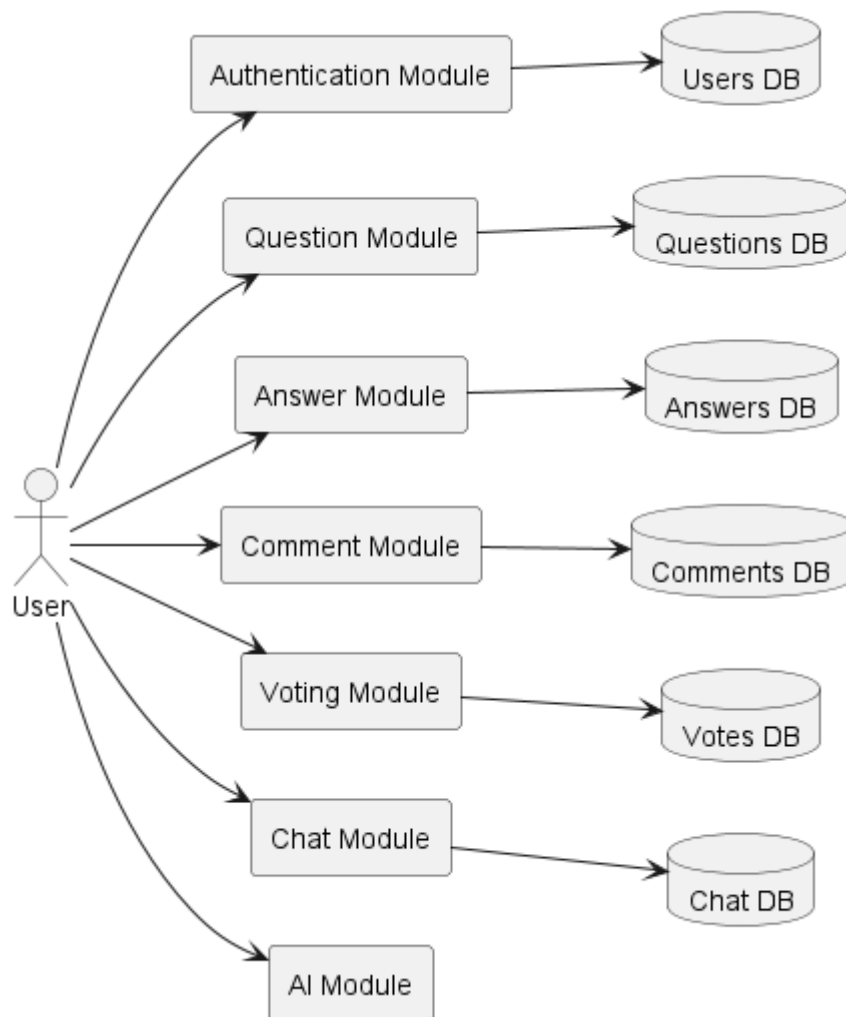


Vote Answer Sequence Diagram

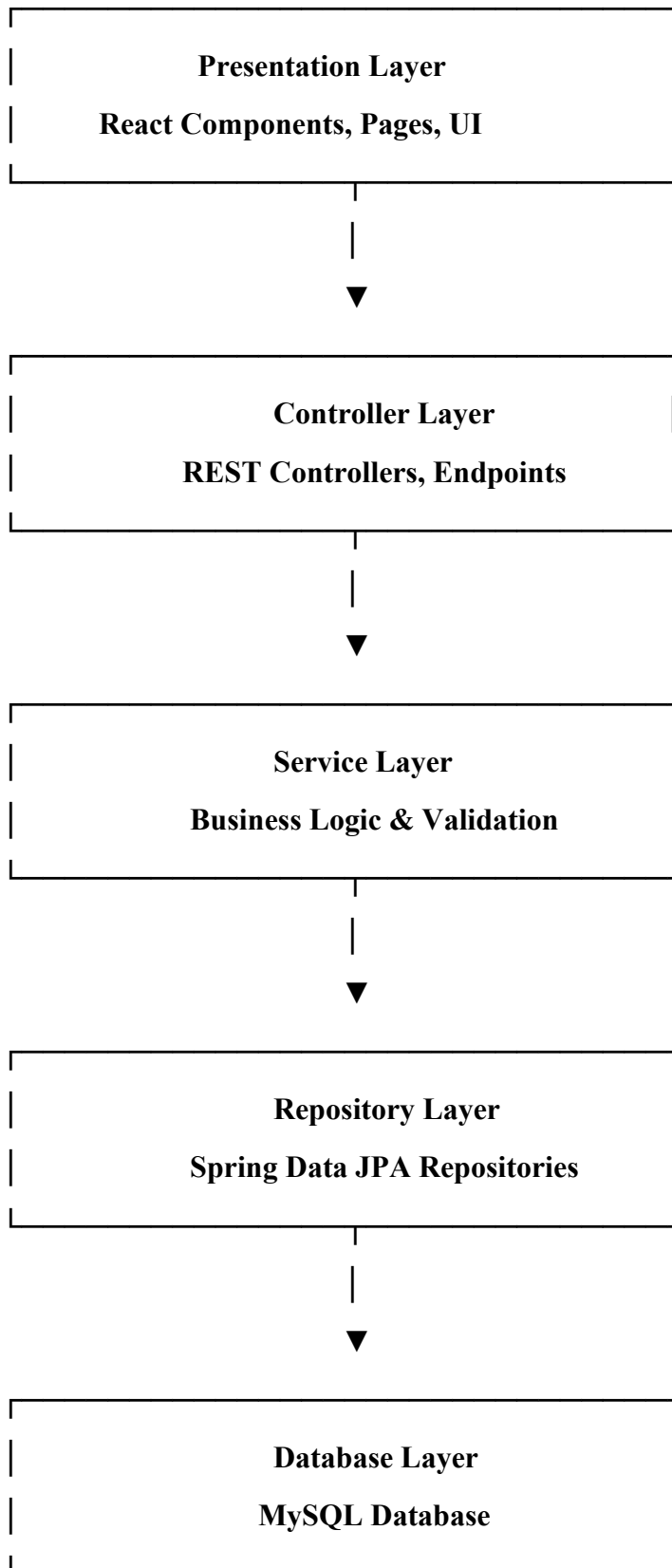




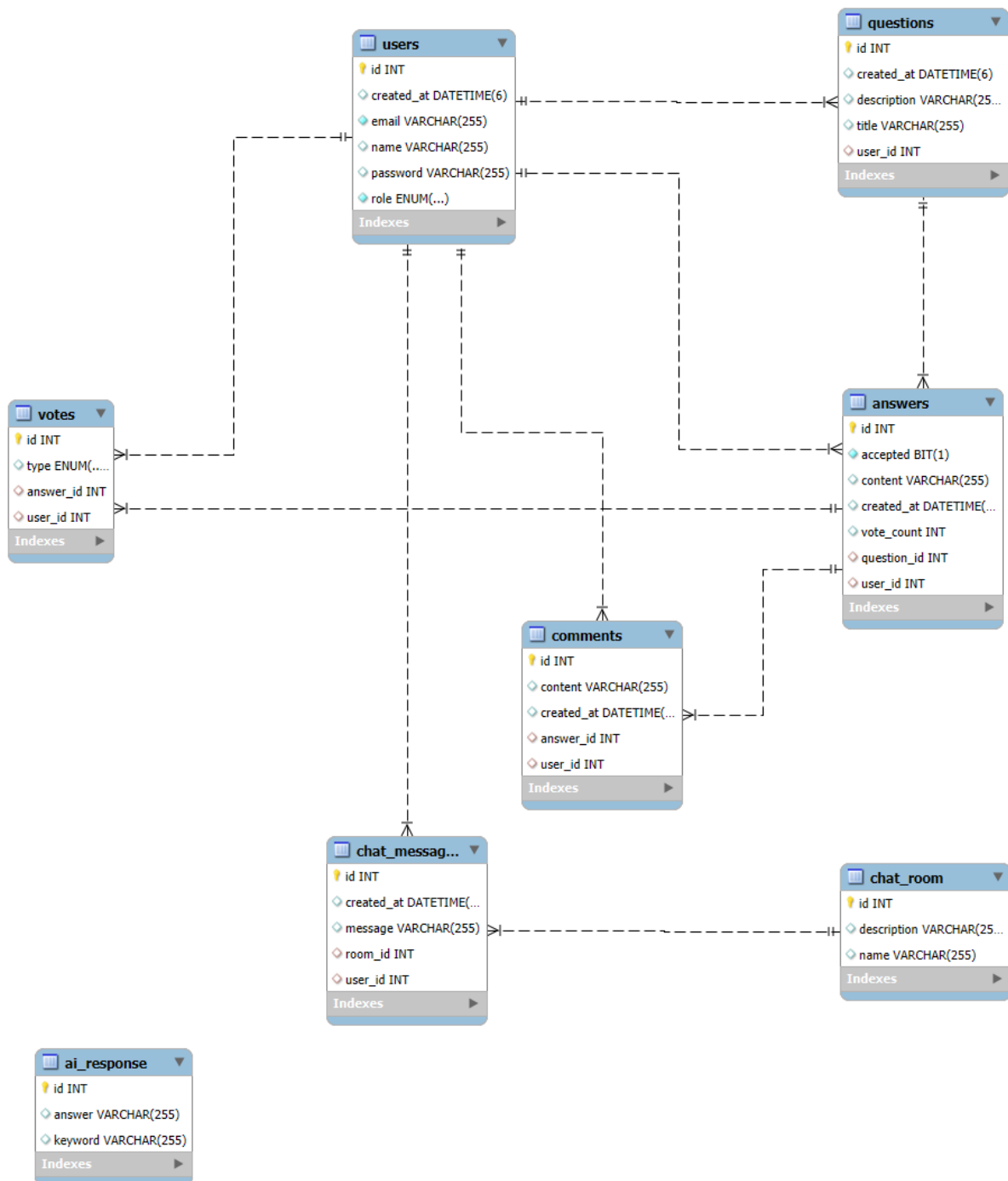
2.5 Data Flow Diagram (DFD)



2.6 Layered Architecture Diagram



2.7 Entity Relationship (ER) Diagram



2.8 Database Design

2.8.1 Users Table

Column Name	Data Type	Constraint
id	INT	PK, AUTO_INCREMENT
name	VARCHAR(255)	NOT NULL
email	VARCHAR(255)	UNIQUE, NOT NULL
password	VARCHAR(255)	NOT NULL
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP
role	VARCHAR(20)	NOT NULL

2.8.2 Questions Table

Column Name	Data Type	Constraint
id	INT	PK, AUTO_INCREMENT
title	VARCHAR(255)	NOT NULL
description	TEXT	NOT NULL
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP
user_id	INT	FK (Users.id)

2.8.3 Answers Table

Column Name	Data Type	Constraint
id	INT	PK, AUTO_INCREMENT
content	TEXT	NOT NULL
vote_count	INT	DEFAULT 0
accepted	BOOLEAN	DEFAULT FALSE
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP
user_id	INT	FK (Users.id)
question_id	INT	FK (Questions.id)

2.8.4 Comments Table

Column Name	Data Type	Constraint
id	INT	PK, AUTO_INCREMENT
content	TEXT	NOT NULL
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP
user_id	INT	FK (Users.id)
answer_id	INT	FK (Answers.id)

2.8.5 Votes Table

Column Name	Data Type	Constraint
id	INT	PK, AUTO_INCREMENT
type	VARCHAR(20)	NOT NULL
user_id	INT	FK (Users.id)
answer_id	INT	FK (Answers.id)
(user_id, answer_id)	-	UNIQUE CONSTRAINT

2.8.6 Chat Rooms Table

Column Name	Data Type	Constraint
id	INT	PK, AUTO_INCREMENT
name	VARCHAR(255)	NOT NULL
description	TEXT	NULL

2.8.7 Chat Messages Table

Column Name	Data Type	Constraint
id	INT	PK, AUTO_INCREMENT
message	TEXT	NOT NULL
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP
user_id	INT	FK (Users.id)
room_id	INT	FK (Chat_Room.id)

2.8.8 Database Relationships

User (1) ---- (M) Question

User (1) ---- (M) Answer

User (1) ---- (M) Comment

User (1) ---- (M) Vote

User (1) ---- (M) ChatMessage

Question (1) ---- (M) Answer

Answer (1) ---- (M) Comment

Answer (1) ---- (M) Vote

ChatRoom (1) ---- (M) ChatMessage

2.8.9 Database Normalisation

First Normal Form (1NF)

The DoConnect AI database satisfies First Normal Form (1NF) because all tables contain atomic values and there are no repeating groups or multi-valued attributes. Each table has a unique primary key (id), and every column stores a single value. For example, the Users table stores one email address, one name, and one role for each user.

Second Normal Form (2NF)

The DoConnect AI database satisfies Second Normal Form (2NF) because all non-key attributes are fully dependent on their respective primary keys. For example, in the Questions table, the attributes title and description depend entirely on question_id. Similarly, in the Answers table, the attribute content depends entirely on answer_id. Therefore, there are no partial dependencies in the database.

Third Normal Form (3NF)

The DoConnect AI database satisfies Third Normal Form (3NF) because there are no transitive dependencies between non-key attributes. Information is stored only in its respective table and relationships are maintained using foreign keys. User details are stored only in the Users table, question details are stored only in the Questions table, and answer details are stored only in the Answers table. This design minimizes data redundancy and maintains data consistency and integrity.

Conclusion

The database design of DoConnect AI follows First Normal Form (1NF), Second Normal Form (2NF), and Third Normal Form (3NF). The normalized structure reduces data redundancy, improves consistency, and ensures efficient data management through the use of primary and foreign key relationships.

2.9 API Documentation

2.9.1 Authentication APIs

POST /users/register – Registers a new user in the system.

POST /users/login – Authenticates the user and returns a JWT token.

2.9.2 Question APIs

POST /questions – Creates a new question.

GET /questions – Retrieves all questions.

GET /questions/{id} – Retrieves a question by ID.

PUT /questions/{id} – Updates a question.

DELETE /questions/{id} – Deletes a question.

2.9.3 Answer APIs

POST /answers – Adds an answer to a question.

GET /answers/question/{questionId} – Retrieves answers for a question.

PUT /answers/{id} – Updates an answer.

DELETE /answers/{id} – Deletes an answer.

PUT /answers/accept/{answerId} – Marks an answer as accepted.

2.9.4 Comment APIs

POST /comments – Adds a comment.

GET /comments/answer/{answerId} – Retrieves comments for an answer.

DELETE /comments/{id} – Deletes a comment.

2.9.5 Vote APIs

POST /votes – Adds an upvote or downvote.

GET /votes/answer/{answerId} – Retrieves votes for an answer.

DELETE /votes/answer/{answerId} – Removes a vote.

2.9.6 Chat Room APIs

POST /chatrooms – Creates a new chat room.

GET /chatrooms – Retrieves all chat rooms.

GET /chatrooms/{id} – Retrieves a chat room by ID.

DELETE /chatrooms/{id} – Deletes a chat room.

2.9.7 Chat Message APIs

POST /chatmessages – Sends a message to a chat room.

GET /chatmessages/room/{roomId} – Retrieves messages from a chat room.

2.10 Deployment Guide

2.10.1 Software Requirements

Software	Version
Operating System	Windows 10/11
Java	JDK 22
Spring Boot	3.x
MySQL	8.0
Node.js	v22.x
Maven	3.9.x
Spring Tool Suite (STS)	Latest
Visual Studio Code	Latest
Postman	Latest
Git	Latest

2.10.2 Backend Setup

1. Clone the project repository.
2. Import the Backend project into Spring Tool Suite.
3. Configure database properties in application.properties.
4. Create a MySQL database named doconnectai.
5. Run the Spring Boot application using:

```
mvn spring-boot:run
```
6. Verify the APIs using Swagger UI or Postman.

2.10.2 Frontend Setup

Open the doconnect-ai-frontend project in Visual Studio Code.

Install the dependencies:

```
npm install
```

Start the application:

```
npm run dev
```

Open the application in the browser:

```
http://localhost:3000
```

2.10.4 Database Setup

Execute the following command:

```
CREATE DATABASE doconnectai;
```

The application uses Spring Data JPA and Hibernate, which automatically generate the required tables during application startup.

Deployment Architecture

The DoConnect AI application follows a three-tier architecture:

Presentation Layer → React Frontend

Business Layer → Spring Boot REST APIs

Data Layer → MySQL Database

The frontend communicates with the backend through REST APIs, and the backend interacts with the MySQL database using Spring Data JPA and Hibernate.

2.11 Folder Structure

Capstone-doconnectai/

```
|
|
|— Backend/
|   |— src/main/java/com/doconnectai
|   |   |— controller
|   |   |— service
|   |   |— repository
|   |   |— entity
|   |   |— dto
|   |   |— mapper
|   |   |— security
|   |   |— exception
|   |   |— config
|   |
|   |— src/main/resources
|   |   |— application.properties
|   |   |— static
|
|— doconnect-ai-frontend/
```



```
| | └─ src
| |   └─ components
| |   └─ pages
| |   └─ services
| |   └─ hooks
| |   └─ context
| |   └─ styles
| |   └─ assets
| |
| └─ public
|
└─ ai-service/
    └─ controller
    └─ service
    └─ dto
    └─ config
    └─
└─ README.md
```

The project follows a modular full-stack architecture consisting of Backend, Frontend, and AI Service modules. The backend follows layered architecture using Controller, Service, Repository, and Entity layers. The frontend follows component-based architecture using React components, hooks, and services for API communication.

CHAPTER 3: TESTING DOCUMENTATION

3.1 Test Cases

Test Case ID	Module	Test Scenario	Expected Result	Actual Result	Status
TC01	Registration	Register with valid details	User account created successfully	User account created successfully	Pass
TC02	Registration	Register with invalid email	Validation error displayed	Validation error displayed	Pass
TC03	Login	Login with valid credentials	JWT token generated and login successful	JWT token generated and login successful	Pass
TC04	Login	Login with invalid password	Invalid credentials message displayed	Invalid credentials message displayed	Pass
TC05	Question	Add a valid question	Question created successfully	Question created successfully	Pass
TC06	Question	Submit empty title	Validation message displayed	Validation message displayed	Pass
TC07	Question	View all questions	Questions displayed successfully	Questions displayed successfully	Pass
TC08	Answer	Add answer to a question	Answer added successfully	Answer added successfully	Pass
TC09	Answer	Accept an answer	Selected answer marked as accepted	Selected answer marked as accepted	Pass
TC10	Comment	Add comment on an answer	Comment added successfully	Comment added successfully	Pass
TC11	Vote	Upvote an answer	Vote count increased	Vote count increased	Pass
TC12	Vote	Downvote an answer	Vote count decreased	Vote count decreased	Pass
TC13	Chat Room	Create a chat room	Chat room created successfully	Chat room created successfully	Pass
TC14	Chat Message	Send a message in chat room	Message sent successfully	Message sent successfully	Pass
TC15	Admin	Delete a question	Question deleted successfully	Question deleted successfully	Pass
TC16	Admin	Delete an answer	Answer deleted successfully	Answer deleted successfully	Pass
TC17	Security	Access protected API without token	Unauthorized response returned	Unauthorized response returned	Pass

TC18	Security	Access API with expired token	Session expired response returned	Session expired response returned	Pass
------	----------	-------------------------------	-----------------------------------	-----------------------------------	------

3.2 Execution Results

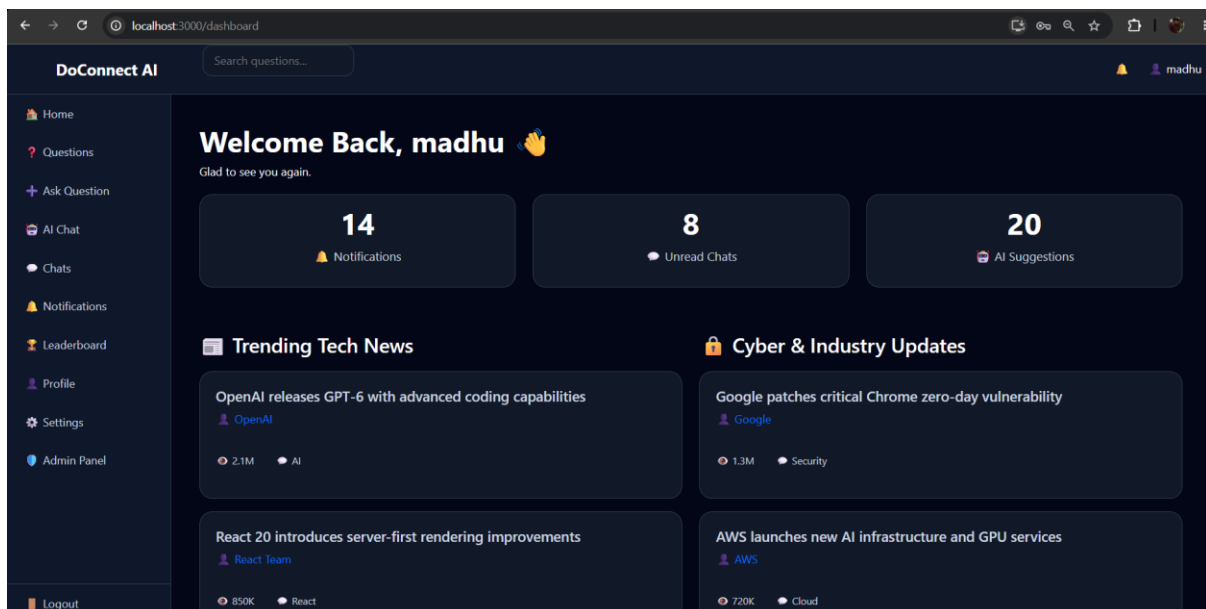
The DoConnect AI application was successfully developed and executed using React, Spring Boot, and MySQL. All major functionalities, including user authentication, question management, answer management, commenting, voting, chat rooms, and administrative operations, were tested successfully.

The application was able to:

- Register and authenticate users using JWT.
- Allow users to post and manage questions and answers.
- Enable commenting and voting functionalities.
- Provide chat room communication features.
- Implement role-based access control for administrators.
- Validate user inputs and handle exceptions effectively.

All implemented functionalities behaved as expected and no critical failures were observed during execution.

Screenshot 1: Dashboard Page



Screenshot 2: Spring Boot Console

```

2020-06-18T07:55:55.821+05:30 INFO 32240 --- [documental] [ main] com.documental.DocumentalApplication : Starting DocumentalApplication using Java 20.0.3 with PID 32240 (C:\View folder (5)
2020-06-18T07:55:55.822+05:30 DEBUG 32240 --- [documental] [ main] com.documental.DocumentalApplication : Running with Spring Boot v3.5.15, Spring v6.2.19
2020-06-18T07:55:55.823+05:30 INFO 32240 --- [documental] [ main] com.documental.DocumentalApplication : No active profile set, falling back to 1 default profile: "default"
2020-06-18T07:55:56.376+05:30 INFO 32240 --- [documental] [ main] s.s.d.r.c.RepositoryConfigurationDelegate : Bootstrapping Spring Data JPA repositories in DEFAULT mode.
2020-06-18T07:55:56.431+05:30 INFO 32240 --- [documental] [ main] s.s.d.r.c.RepositoryConfigurationDelegate : Finished Spring Data repository scanning in 47 ms. Found 7 JPA repository interfaces
2020-06-18T07:55:56.887+05:30 INFO 32240 --- [documental] [ main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat initialized with port 8080 (http)
2020-06-18T07:55:56.905+05:30 INFO 32240 --- [documental] [ main] o.apache.catalina.core.StandardService : Starting service [Tomcat]
2020-06-18T07:55:56.905+05:30 INFO 32240 --- [documental] [ main] o.apache.catalina.core.StandardEngine : Starting Servlet engine: [Apache Tomcat/10.1.55]
2020-06-18T07:55:56.962+05:30 INFO 32240 --- [documental] [ main] o.a.e.c.c.[Tomcat].[localhost].[/] : Initializing Spring embedded WebApplicationContext
2020-06-18T07:55:56.963+05:30 INFO 32240 --- [documental] [ main] w.s.c.ServletWebServerApplicationContext : Root WebApplicationContext: initialization completed in 1086 ms
2020-06-18T07:55:57.037+05:30 INFO 32240 --- [documental] [ main] com.documental.service.JwtFilter : Filter 'JwtFilter' configured for use
2020-06-18T07:55:57.141+05:30 INFO 32240 --- [documental] [ main] o.hibernate.jpa.internal.util.LogHelper : HH0000204: Processing PersistenceUnitInfo [name: default]
2020-06-18T07:55:57.197+05:30 INFO 32240 --- [documental] [ main] org.hibernate.Version : HH0000412: Hibernate ORM core version 6.6.53.Final
2020-06-18T07:55:57.234+05:30 INFO 32240 --- [documental] [ main] o.h.e.internal.RegionFactoryInitiator : HH0000266: Second-level cache disabled
2020-06-18T07:55:57.495+05:30 INFO 32240 --- [documental] [ main] o.s.o.j.p.SpringPersistenceUnitInfo : No LoadTimeWeaver setup; ignoring JPA class transformer
2020-06-18T07:55:57.543+05:30 INFO 32240 --- [documental] [ main] com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Starting...
2020-06-18T07:55:57.819+05:30 INFO 32240 --- [documental] [ main] com.zaxxer.hikari.pool.HikariPool : HikariPool-1 - Added connection com.mysql.cj.jdbc.ConnectionImpl@6c903998
2020-06-18T07:55:57.821+05:30 INFO 32240 --- [documental] [ main] com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Start completed.
2020-06-18T07:55:57.898+05:30 INFO 32240 --- [documental] [ main] org.hibernate.orm.connections.pooling : HH0000005: Database info:
Database JDBC URL [connecting through datasource 'HikariDataSource (HikariPool-1)']
Database driver: undefined/unknown
Database version: 8.0.43
Autocommit mode: undefined/unknown
Isolation level: undefined/unknown
Minimum pool size: undefined/unknown
Maximum pool size: undefined/unknown
2020-06-18T07:55:58.708+05:30 INFO 32240 --- [documental] [ main] o.h.e.t.j.p.i.JtaPlatformInitiator : HH0000409: No JTA platform available (set 'hibernate.transaction.jta.platform' to e
2020-06-18T07:55:58.792+05:30 INFO 32240 --- [documental] [ main] j.localContainerEntityManagerFactoryBean : Initialized JPA EntityManagerFactory for persistence unit 'default'
2020-06-18T07:55:59.157+05:30 WARN 32240 --- [documental] [ main] jpaBaseConfiguration$JpaMetaConfiguration : spring.jpa.open-in-view is enabled by default. Therefore, database queries may be p
2020-06-18T07:55:59.184+05:30 WARN 32240 --- [documental] [ main] s.s.UserDetailsServiceAutoConfiguration :
Using generated security password: 64948a10-f81f-4140-84af-2498996c870a
This generated password is for development use only. Your security configuration must be updated before running your application in production.
2020-06-18T07:55:59.190+05:30 INFO 32240 --- [documental] [ main] r.InitializeUserDetailsServiceConfigurer : Global AuthenticationManager configured with UserDetailsService bean with name inMe
2020-06-18T07:55:59.631+05:30 INFO 32240 --- [documental] [ main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on port 8080 (http) with context path '/'
2020-06-18T07:55:59.640+05:30 INFO 32240 --- [documental] [ main] com.documental.DocumentalApplication : Started DocumentalApplication in 4.129 seconds (process running for 4.873)

```

Screenshot 3: MySql Database

```
mysql> use doconnectdb;
Database changed
mysql> show tables;
+-----+
| Tables_in_doconnectdb |
+-----+
| ai_response            |
| answers                |
| chat_messages          |
| chat_room              |
| comments               |
| questions              |
| users                  |
| votes                  |
+-----+
8 rows in set (0.00 sec)
```

Screenshot 4: JUNIT Test Results

The screenshot displays an IDE with the following components:

- Package Explorer:** Shows the project structure with packages like `com.dconnectai.application` and `com.dconnectai.service`.
- Source Code:** The `ListCommentDto` and `ListComment` classes are visible. `ListCommentDto` has a `getCommentsByAnswer_Id` method. `ListComment` has a `findByAnswer_Id` method.
- Problems:** A warning is shown for the deprecated `getCommentsByAnswer_Id` method in `ListCommentDto`.
- Console:** Shows the application running successfully. The output includes:
 - Initializing Spring DispatcherServlet
 - Initializing Servlet 'dispatcherServlet'
 - Completed initialization in 2 ms
 - POST /login successful
 - Valid Token for: madhu@gmail.com,role: ADMIN
 - Fetching user with email: madhu@gmail.com
 - User found with email: madhu@gmail.com

3.3 Defect Summary

Defect ID	Description	Resolution	Status
D01	Username not displayed in Answer Management page	Added userName field in AnswerDto and mapper	Closed
D02	Answer count in Admin Dashboard always displayed as 0	Added API call to load all answers dynamically	Closed
D03	Vote count API returned HTTP 500 error	Corrected endpoint mapping and service implementation	Closed
D04	Validation messages were not shown on frontend	Integrated SweetAlert error handling	Closed
D05	JWT token expiry did not log out users automatically	Implemented Axios interceptor for invalid tokens	Closed
D06	Delete Answer API returned Internal Server Error	Corrected foreign key handling and repository logic	Closed
D07	Comments section UI alignment issue	Updated CSS and component structure	Closed

References

1. <https://spring.io/projects/spring-boot>
2. <https://react.dev>
3. <https://docs.oracle.com/en/java/>
4. <https://dev.mysql.com/doc/>
5. <https://spring.io/projects/spring-security>
6. <https://jwt.io>
7. <https://swagger.io>
8. <https://developer.mozilla.org>
9. <https://www.postman.com>
10. Great Learning Course Materials